

# Natural Language Processing (NLP)

**Mauricio Cerda**

**Nahuel Gómez**

# Motivación: Área relacionada a múltiples problemas

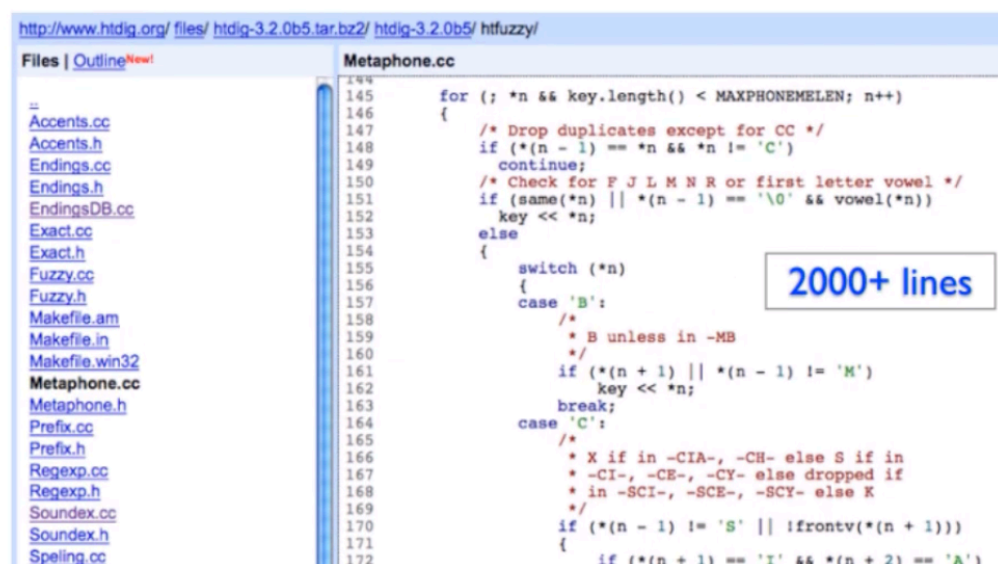
---

- **Modelación de texto (¿De qué habla este texto?).**
- **Análisis de sentimiento (¿Categorías de textos?).**
- Named Entity Recognition (NER) (Ex. nombres de personas).
- Part-of-speech: identificar sustantivos, verbos, adjetivos.
- Respuestas automáticas (!).
- Speech recognition (SR).
- Texto a Habla o Habla a texto.
- Modelación del lenguaje.
- Traducción.

# Motivación: ¿Por qué ML y no sólo simples reglas?

- Reglas (regex) Sólo para casos limitados:
  - ✱ Diferencias entre hablantes.
  - ✱ Evolución constante y bugs recurrentes!!.
  - ✱ Complejo.

[Peter Norvig, <https://norvig.com/spell-correct.html>]



```
145 for (; *n && key.length() < MAXPHONEMELLEN; n++)
146 {
147     /* Drop duplicates except for CC */
148     if (*(n - 1) == *n && *n != 'C')
149         continue;
150     /* Check for F J L M N R or first letter vowel */
151     if (same(*n) || *(n - 1) == '\0' && vowel(*n))
152         key << *n;
153     else
154     {
155         switch (*n)
156         {
157             case 'B':
158                 /* B unless in -MB
159                 */
160                 if (*(n + 1) || *(n - 1) != 'M')
161                     key << *n;
162                 break;
163             case 'C':
164                 /*
165                 * X if in -CIA-, -CH- else S if in
166                 * -CI-, -CE-, -CY- else dropped if
167                 * in -SCI-, -SCE-, -SCY- else K
168                 */
169                 if (*(n - 1) != 'S' || !frontv(*(n + 1)))
170                 {
171                     if (*(n + 1) == 'I' && *(n + 2) == 'A')
```

```
import collections, re
def words(text): return re.findall('[a-z]+', text.lower())

WORDS = collections.Counter(words(file('big.txt').read()))
alphabet = 'abcdefghijklmnopqrstuvwxyz'

def edits1(word):
    splits = [(word[:i], word[i:]) for i in range(len(word) + 1)]
    deletes = [a + b[1:] for a, b in splits if b]
    transposes = [a + b[1] + b[0] + b[2:] for a, b in splits if len(b)>1]
    replaces = [a + c + b[1:] for a, b in splits for c in alphabet if b]
    inserts = [a + c + b for a, b in splits for c in alphabet]
    return set(deletes + transposes + replaces + inserts)

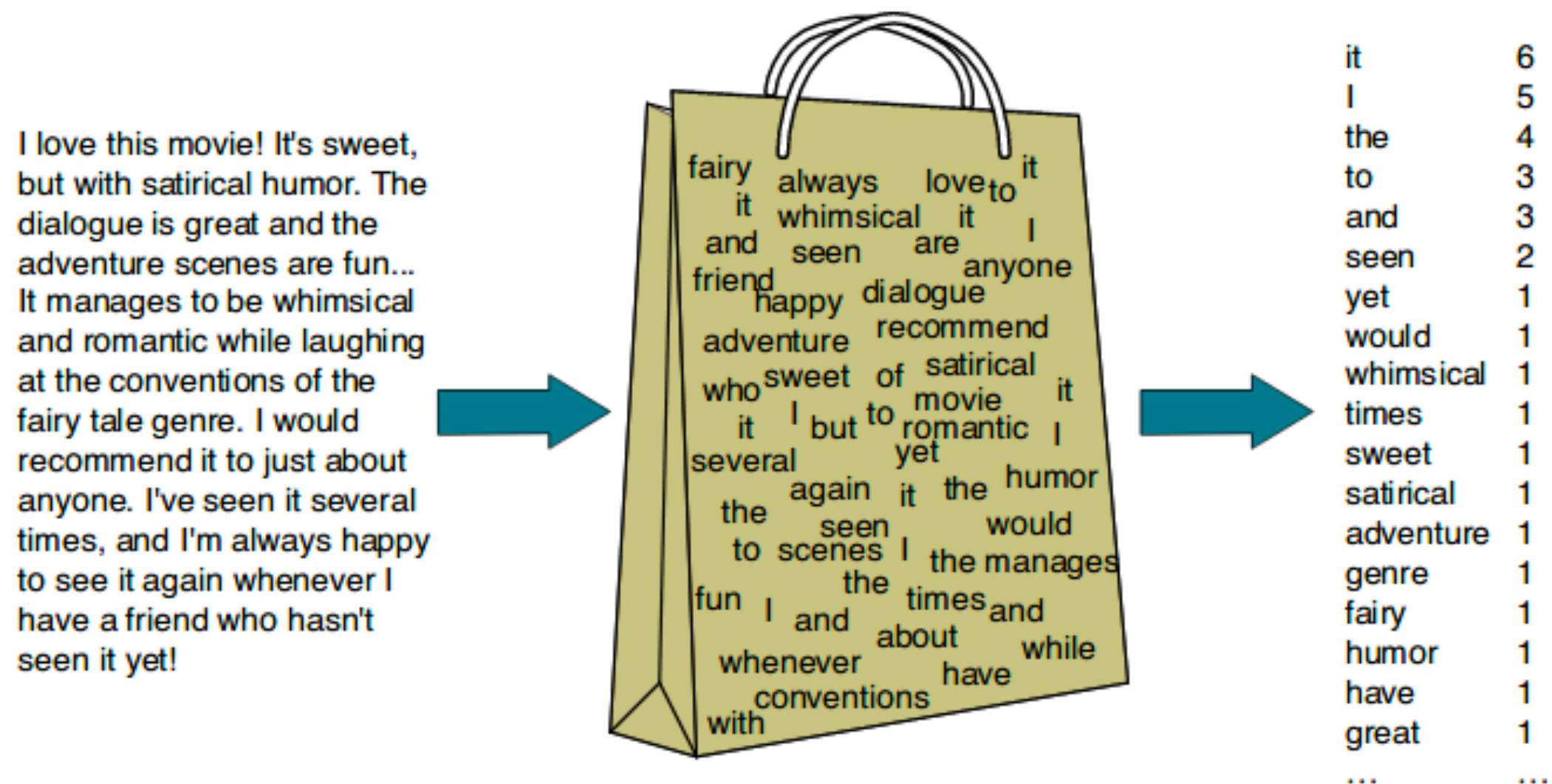
def known_edits2(word):
    return [e2 for e1 in edits1(word) for e2 in edits1(e1) if e2 in WORDS]

def known(words): return [w for w in words if w in WORDS]
```

17 lines

# Bag of words (BoW): idea

- La forma más simple es representar las palabras a través de frecuencias.
- Funciona en textos largos: tópicos definidos (e.g. noticias, spam/not spam).





# Bag of words (BoW): algunos ejemplos

- Mantener solo la raíz de las palabras (Ej. Rapid = Rapidez, Rápido, Rápidamente)
- Manejar menciones de palabras positivas/negativas (fractura, no-fractura)



- ¿Qué texto habrá generado esta nube de palabras?
- Corregir la frecuencia por clase (TF-IDF).

# Bag of words (BoW): TF-IDF

---

- Si tenemos documentos (filas) y palabras (columnas) para todos los textos de Shakespeare:

|                | batalla | <b>bueno</b> | tonto | ingenio |
|----------------|---------|--------------|-------|---------|
| Como gustéis   | 1       | <b>114</b>   | 36    | 20      |
| Noche de reyes | 0       | <b>80</b>    | 58    | 15      |
| Julio César    | 7       | <b>62</b>    | 1     | 2       |
| Enrique V      | 13      | <b>89</b>    | 4     | 3       |

- Algunas palabras están en todos los documentos, podemos asumir que son palabras irrelevantes o que no aportan mayor información (NLP stopwords)

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \log \left( \frac{|D|}{|\{d \in D : t \in d\}|} \right)$$

$\text{tf}(f, d)$  : Frecuencia del término 't' en el documento 'd'

$|D|$  : Número de documentos

$|\{d \in D : t \in d\}|$  : Cantidad de documentos en el corpus con el término t

# Bag of words (BoW): TF-IDF

- Podemos ponderar la frecuencia según qué tan únicas son las palabras en cada documento

|                | batalla | bueno | tonto  | ingenio |
|----------------|---------|-------|--------|---------|
| Como gustéis   | 0.074   | 0     | 0.019  | 0.049   |
| Noche de reyes | 0       | 0     | 0.021  | 0.044   |
| Julio César    | 0.22    | 0     | 0.0036 | 0.018   |
| Enrique V      | 0.28    | 0     | 0.0083 | 0.022   |

- Nos permite eliminar algunas palabras (stopwords).



# BoW: desventajas

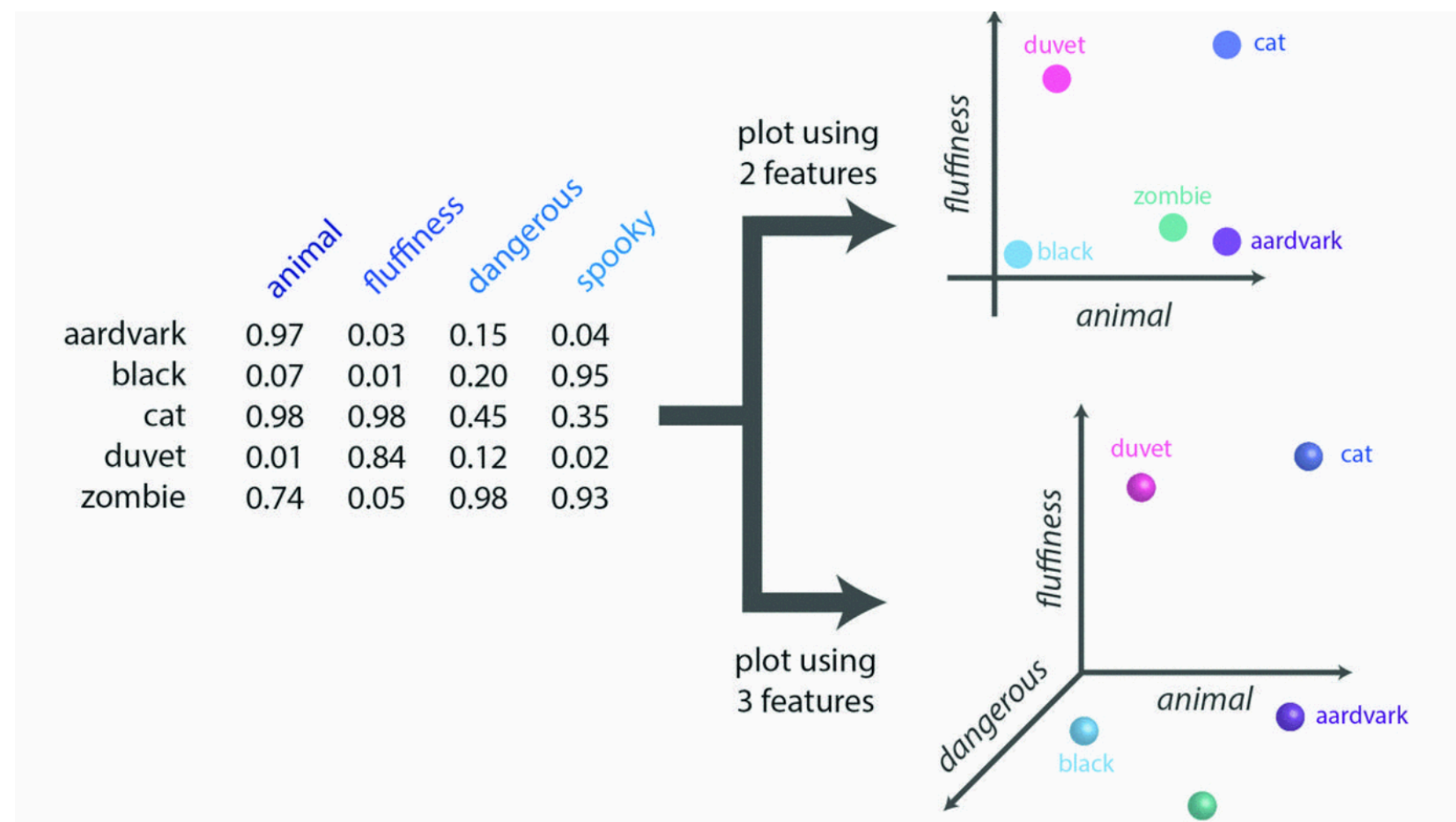
---

- Ignora el orden de las palabras.
- Ignora sinónimos.
- Ignora la polisemia (dependiente del contexto).
- Ignora las ironías.
- Etc...

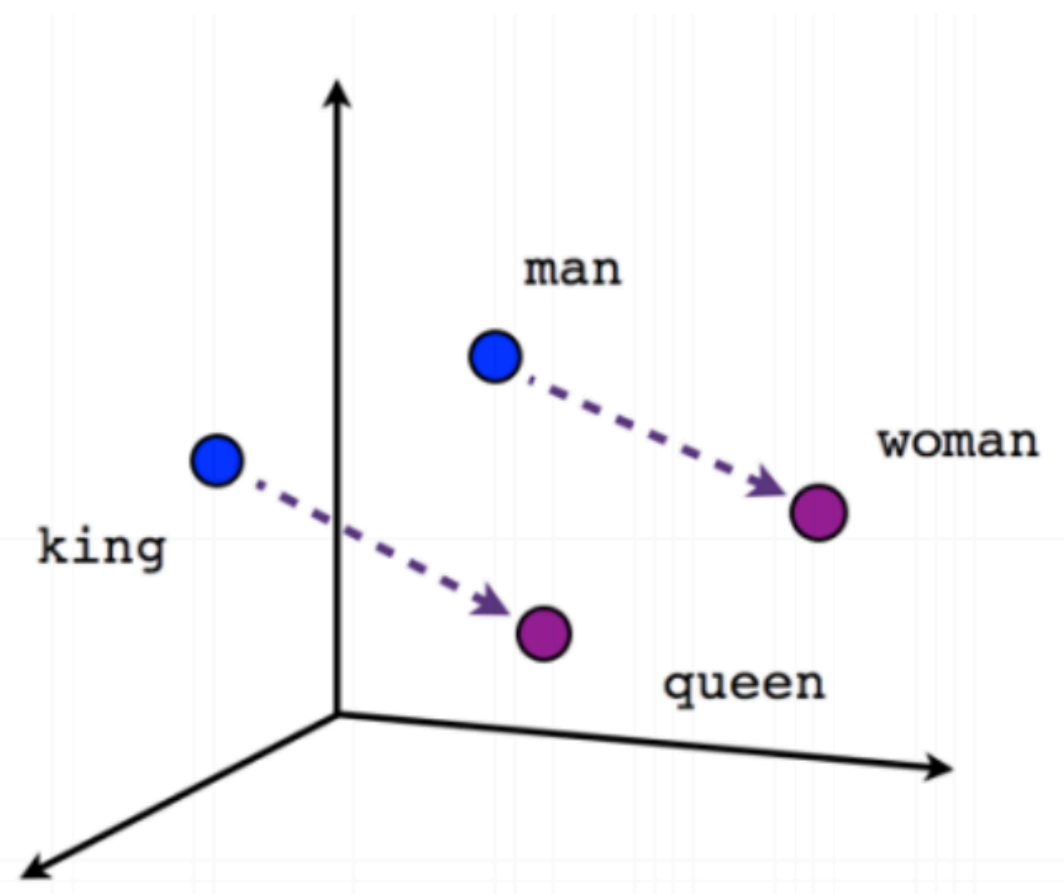


# Words/sentence embeddings (Emb): ¿por qué?

- La frecuencia de palabras no mantiene la semántica intrínseca en cada oración.
- Además contiene muchos valores nulos (sparse). *Un perro tiene más similitudes con un gato que con un auto.*
- Queremos que las representaciones de palabras y las oraciones tengan una lógica semántica (además de densidad).



# Emb: idea



$$\text{Similitud}(u, v) = \cos(\theta) = \frac{u \cdot v}{\|u\| \|v\|}$$

- Cada vector representa un token (en este caso, cada token es una palabra).
- Similitud coseno es un aliado para este tipo de análisis.
- Cuál sería el resultado de -> **King - Man + Queen = ????**

# Emb: Caso de aplicación

---

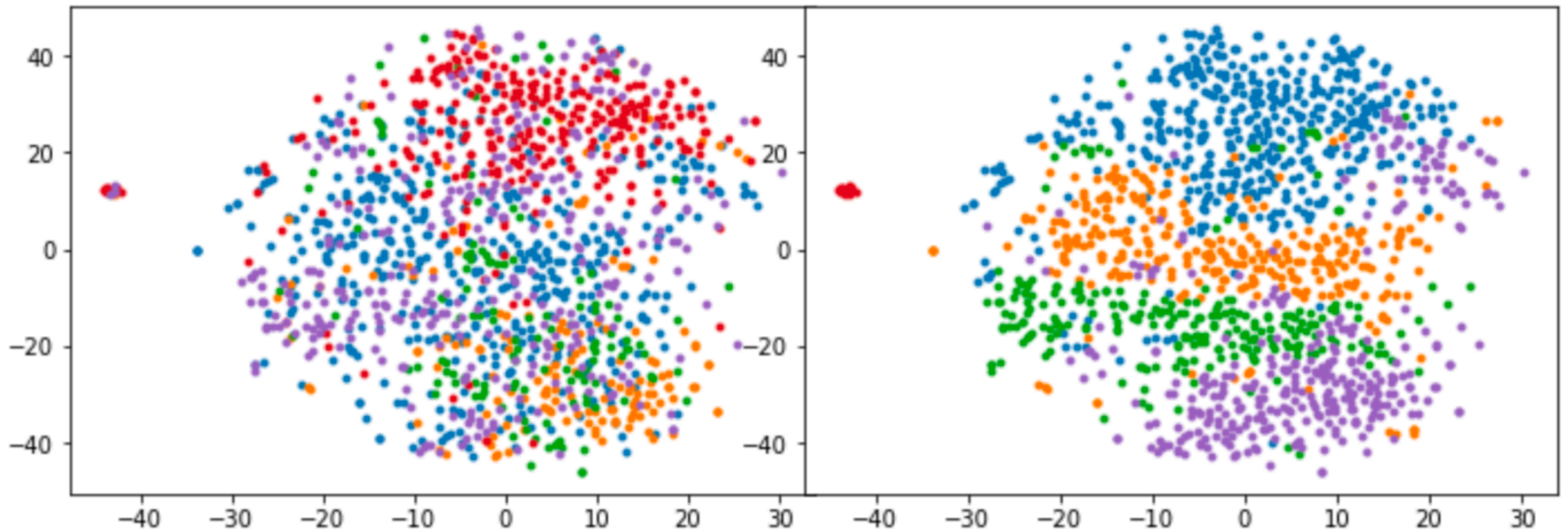
- Una empresa privada hace un llamado a sus internos para que den ideas de innovación
- Llegaron más de 3000 propuestas, algunas con (muchas) faltas de ortografía.

"el desarrollo de una propuesta cultural privada para a cojer la mas ampria bariedade de expresiones que conformas las artes de mi comunidad de mejillones es mi motibante ya que oficia es dar aconocer y el dar aconocer como se hace una empa nada de marisco a la mejillonina o hacer un cuadro deo leo una acuarela ect hacer



# Emb: aplicación con clusters

- Se pueden hacer gráficos con las representaciones de las propuestas, haciendo previa reducción de dimensiones (t-SNE).
- 1 punto = 1 propuesta.



Classification provided by users

Automatic topic clustering

# Emb: aplicación con clusters

- Identificando los clusters, podemos seleccionar los centroides de c/u.
- Podemos nombrar los clusters, por ejemplo, usando visualización de frecuencia (o TF-IDF)



"el desarrollo de una propuesta cultural privada para a cojer la mas ampria bariedade de expresiones que conformas las artes de mi comunidad de mejillones es mi motibante ya que oficia es dar aconocer y el dar aconocer como se hace una empa nada de marisco a la mejillonina o hacer un cuadro deo leo una acuarela ect hacer collares con conchitas de las pla yas artesanias populares taller de titeres de niños y para adultos o tejer con crochet en hilo o el reparar una ret de pes cador etc y así podriamos tener miles de experiencias que conforman una cultura y una identidad comunal la

cual no se agota y menos en una casa de la cultura fija si la casa de oficios tiene la posibilidad con sus talleres bajo do mos de vivos colores a las caletas durante todo el año y lo grar que se realicen pequeños trabajos manuales y con el gusto de mostrarlo si da venderlo trocarlo y si lo desea regalarlo lo magico es que se a transmitido a otro y la comunidad una experiencia unica"

*Niños y cultura.*

# ¿Podemos representar el significado de una palabra?

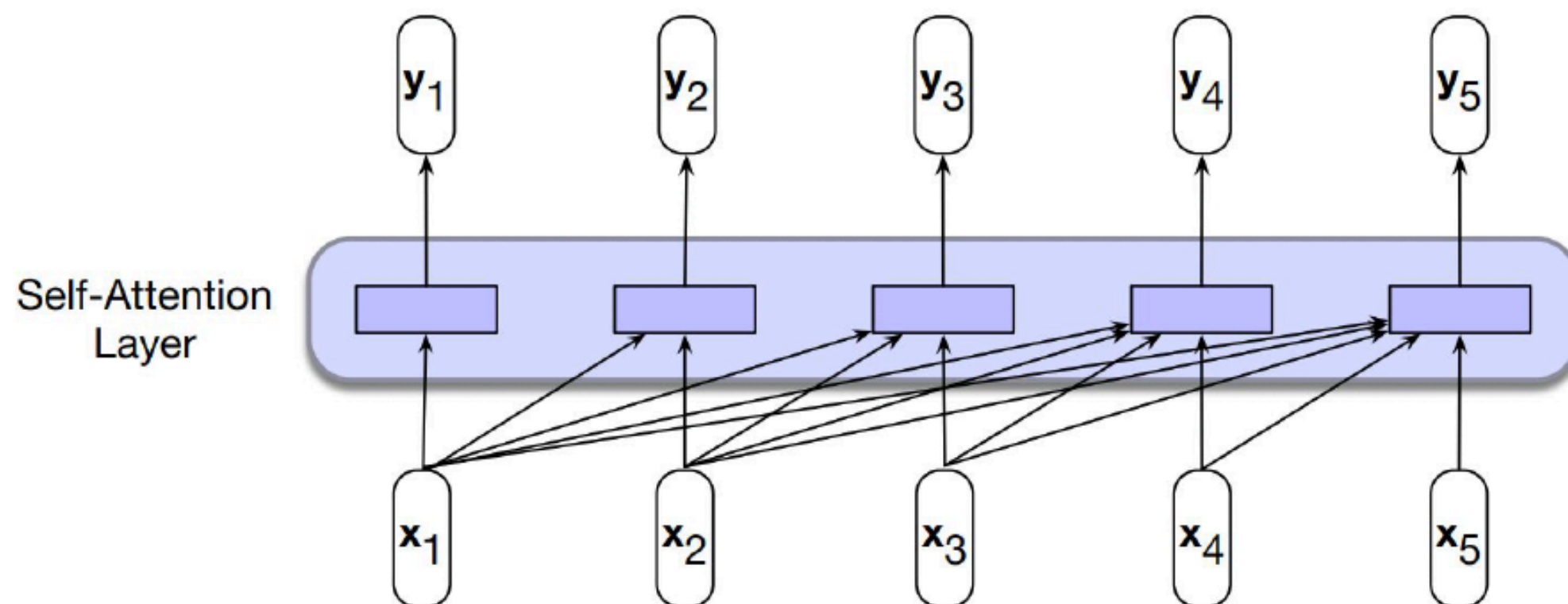
---

- ¿Cómo modelar que el significado de una palabra puede depender de un contexto arbitrariamente largo?
- ¿Cómo evitar que las palabras más cercanas sean necesariamente mas relevantes?
- ¿Cómo modelar múltiples niveles de significado? (Estructura gramatical, semántica, etc)
- ¿Cómo contar con métodos que sean altamente paralelizables? (Evitando métodos recurrentes)

# Modelos Transformers

---

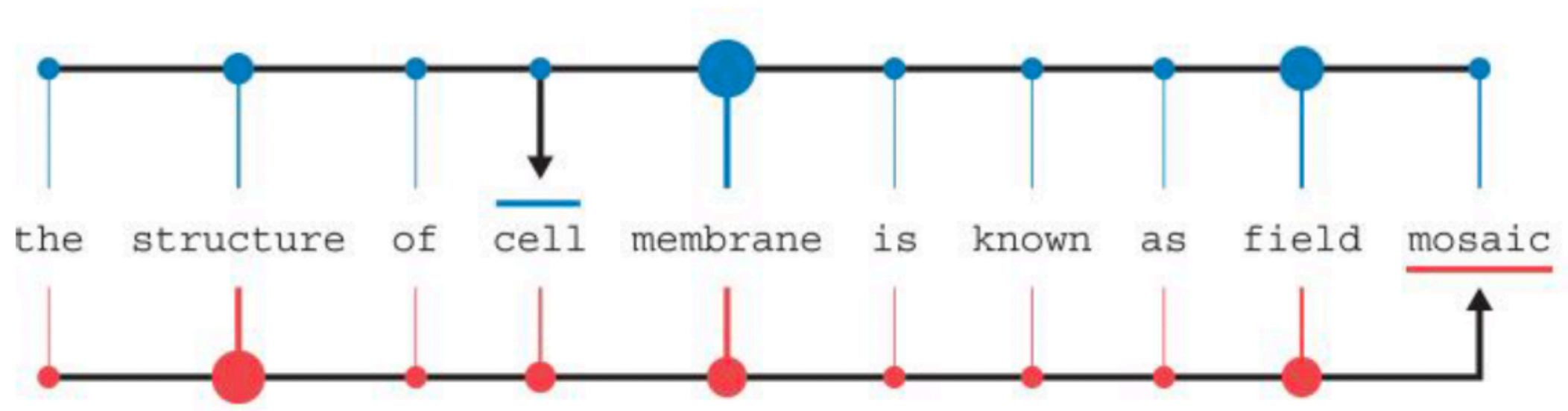
- Conocen al menos uno GPT.
- Se basan en la idea de auto-atención



# Modelos Transformers: auto-atención

---

- Mezcla vectores de entrada para contextualizar la salida
- La mezcla se basa en la relevancia para el vector de entrada.
- Se utiliza en múltiples niveles (Multi-cabezal)

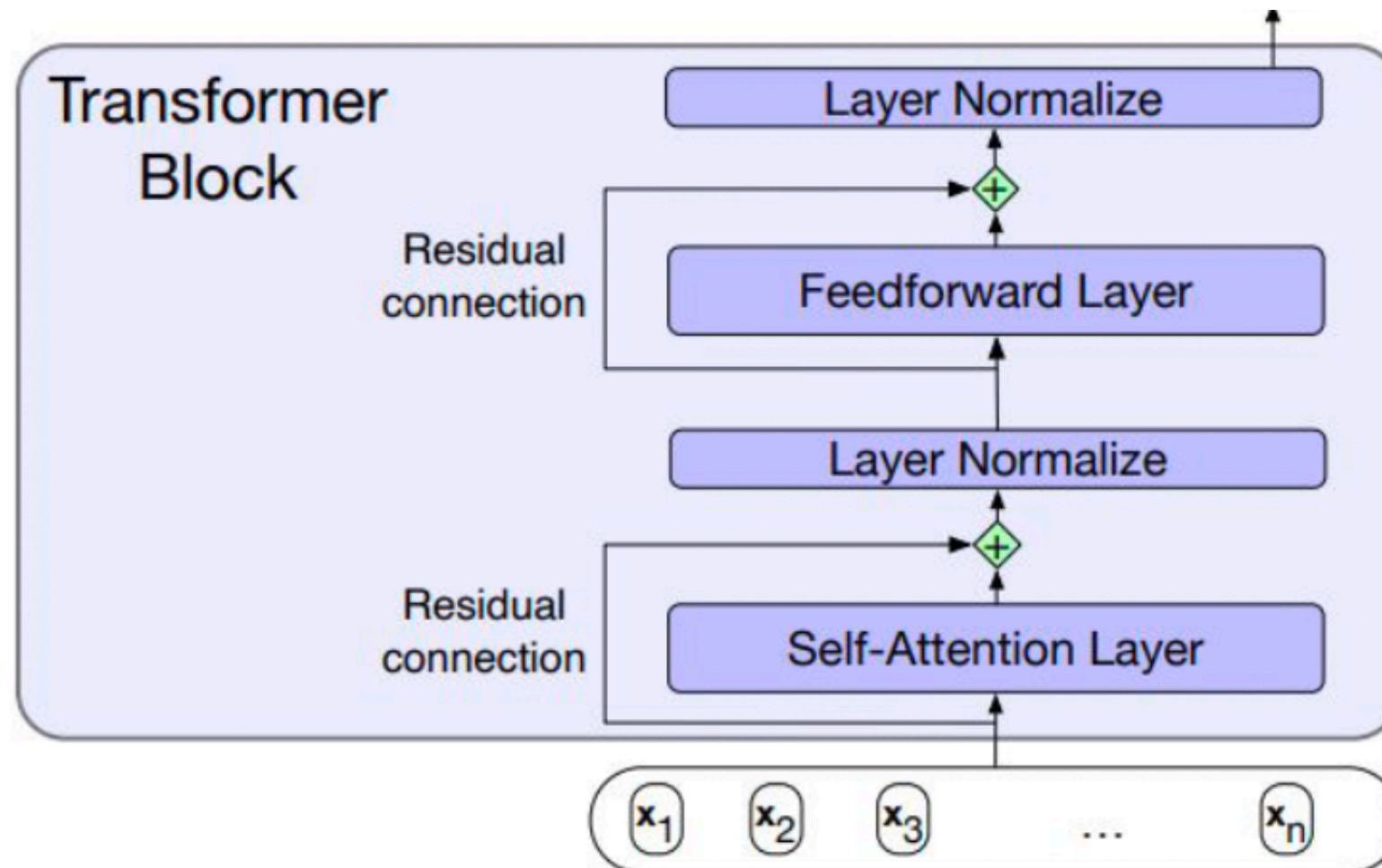




# Modelos Transformers: transformer block

---

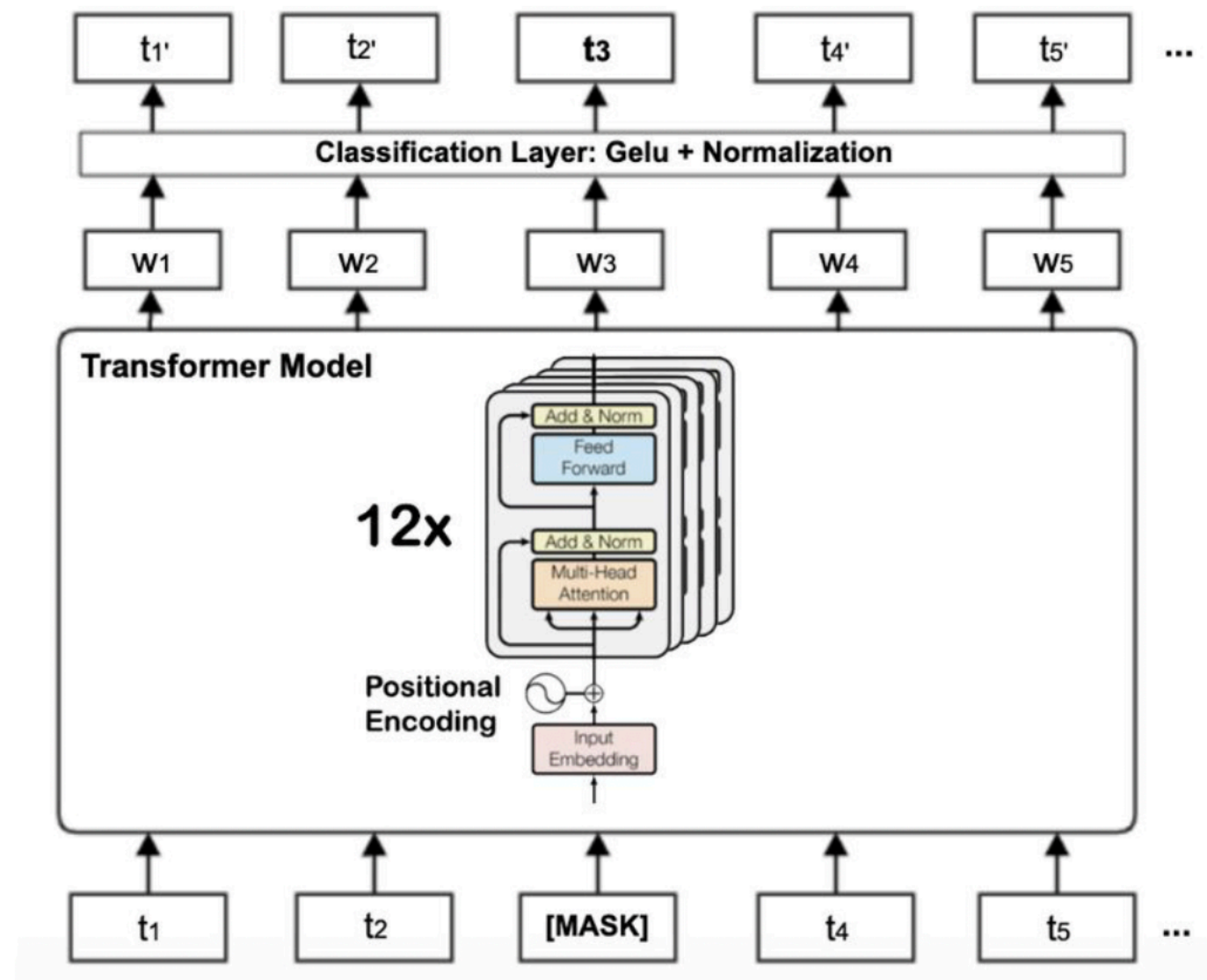
- La auto-atención multicabezal (Multi-Head Self-Attention) es el módulo central de un transformer.
- Esto se combina con otros bloques como normalización





# Modelos Transformers: BERT

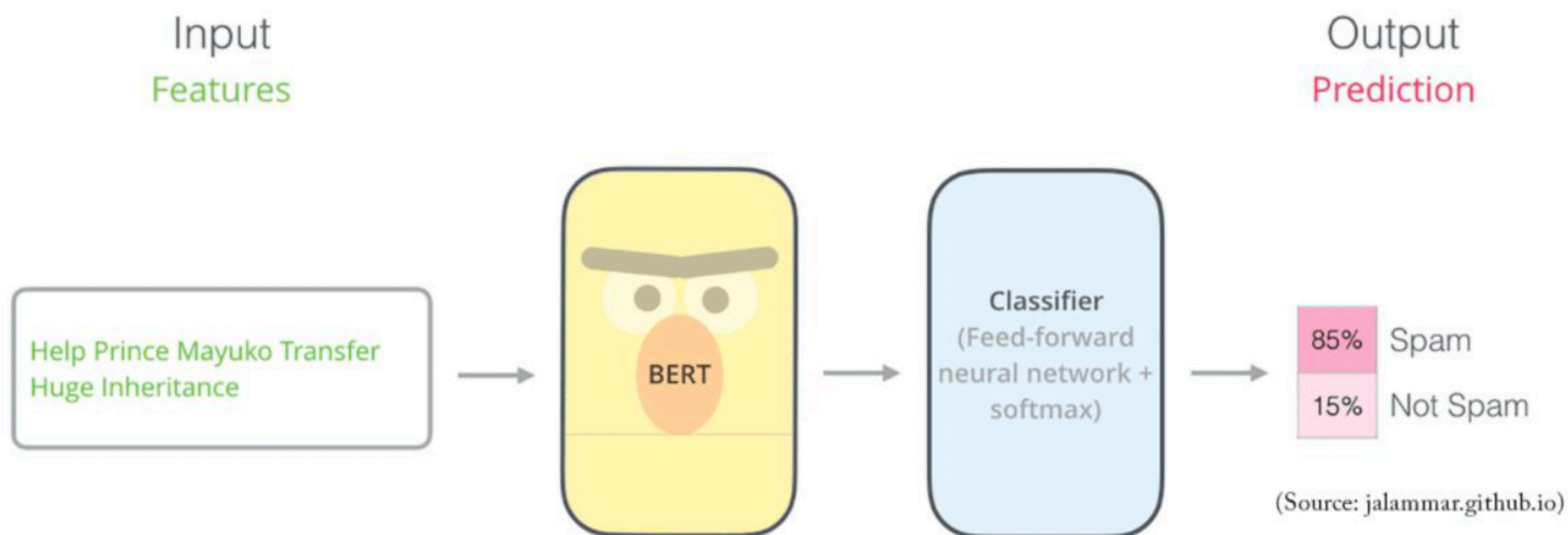
- Bi-directional Encoder Representation Transformer (BERT) es de la familia encoder.
- Permite obtener información de la secuencia a través de la tarea de predecir una palabra enmascarada.



# BERT aplicado (afinado)

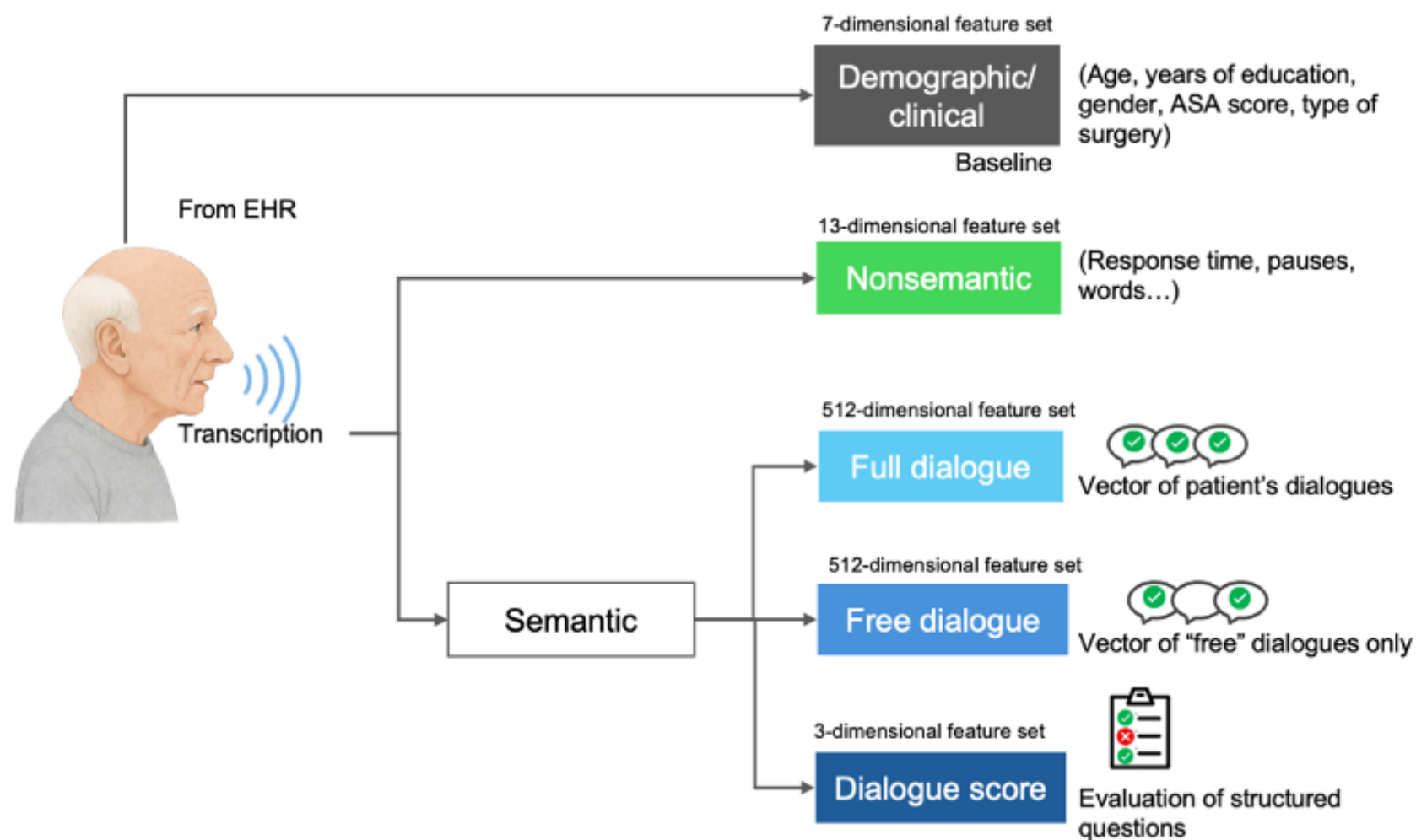
---

- Utilizar el modelo de lenguaje pre-entrenado para una tarea específica.
- Se añaden capas al modelo (se fija el resto) para nuestra tarea.



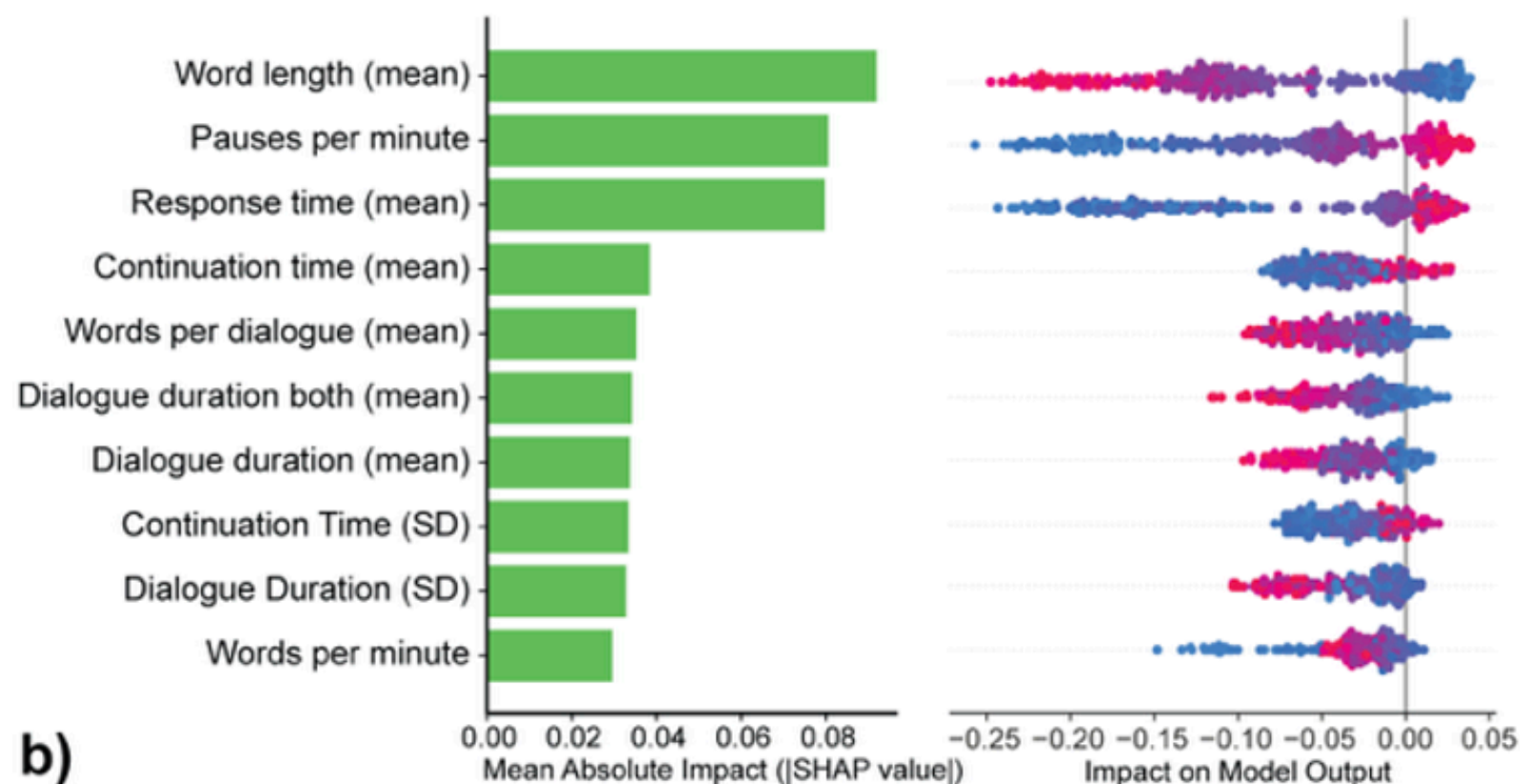
# Emb: aplicación clínica

- Delirium es una alteración aguda de la conciencia.
- La medición actual es el CAM, donde algunas de las variables de medición indican al lenguaje como potencial clasificador.
- Se propone el lenguaje como un biomarcador capaz de diferenciar pacientes sanos de Delirium+ (Gómez, 2025).



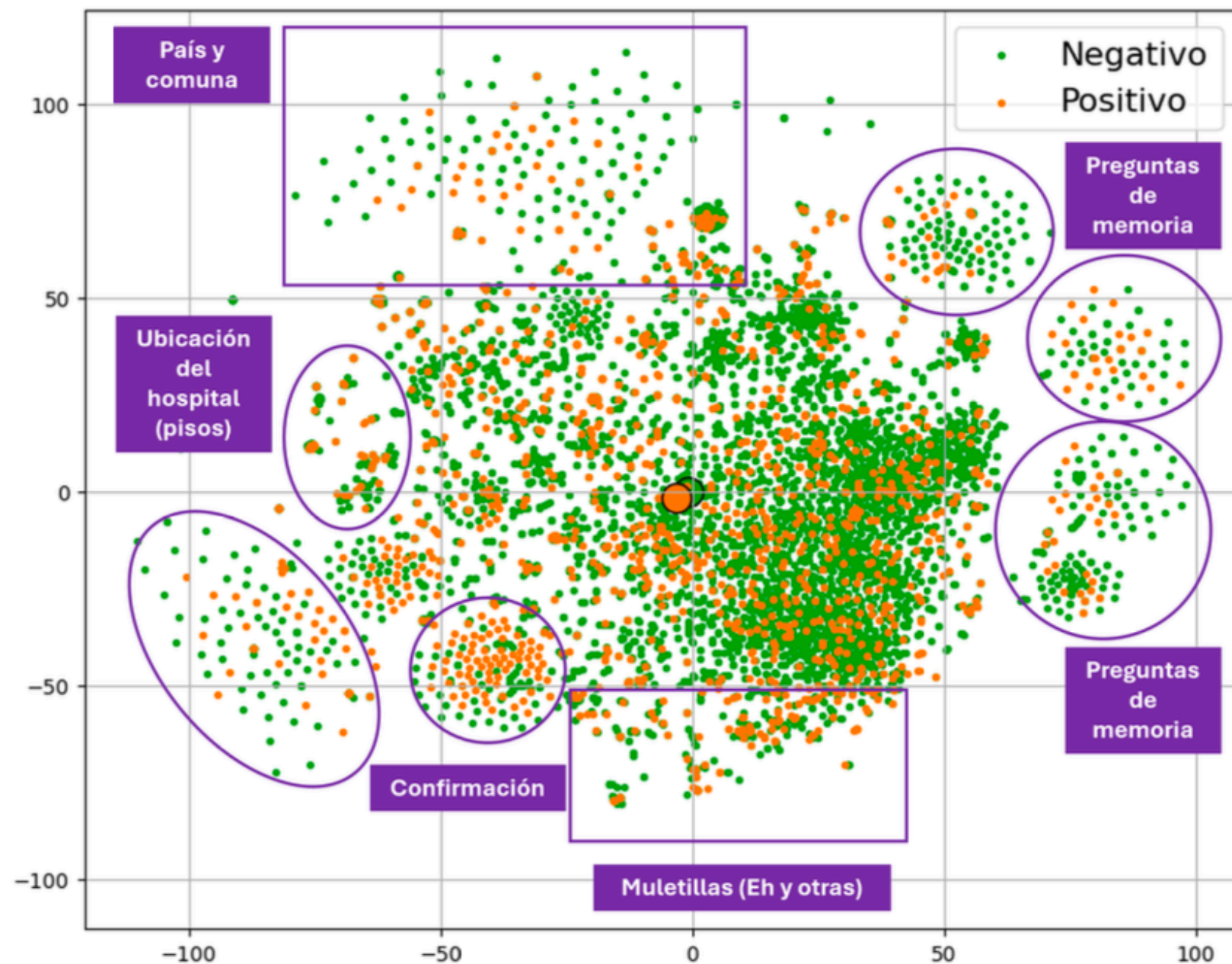
# Emb: Aplicación no semántica

- Con la transcripción de los diálogos junto a los tiempos, podemos calcular tiempo de respuesta, duración del diálogo, entre otras...
- Aplicando un modelo simple (Random Forest) podemos ver la importancia de estas variables.



# Emb: Aplicación semántica

- Usando Beto (Bert español). Se pueden visualizar las preguntas para ver como se distribuyen entre pacientes (t-sne)

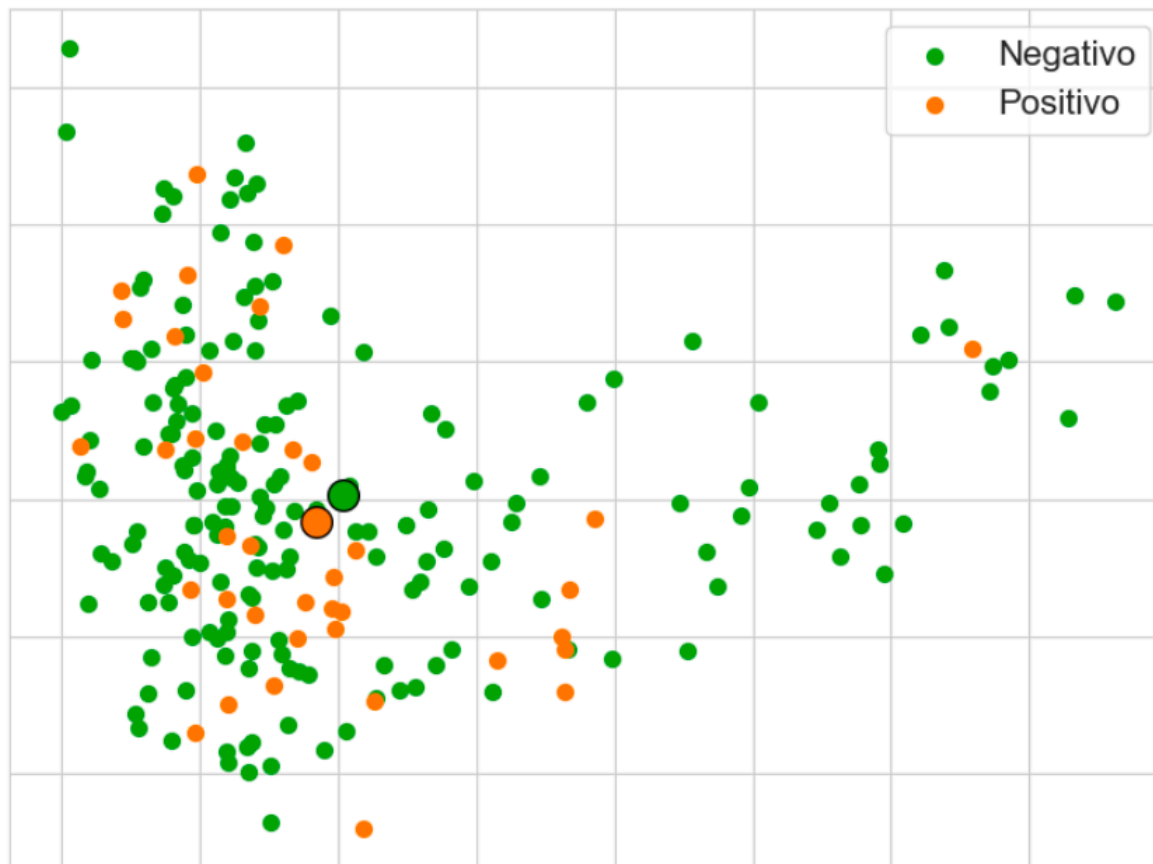




# Emb: Aplicación semántica

---

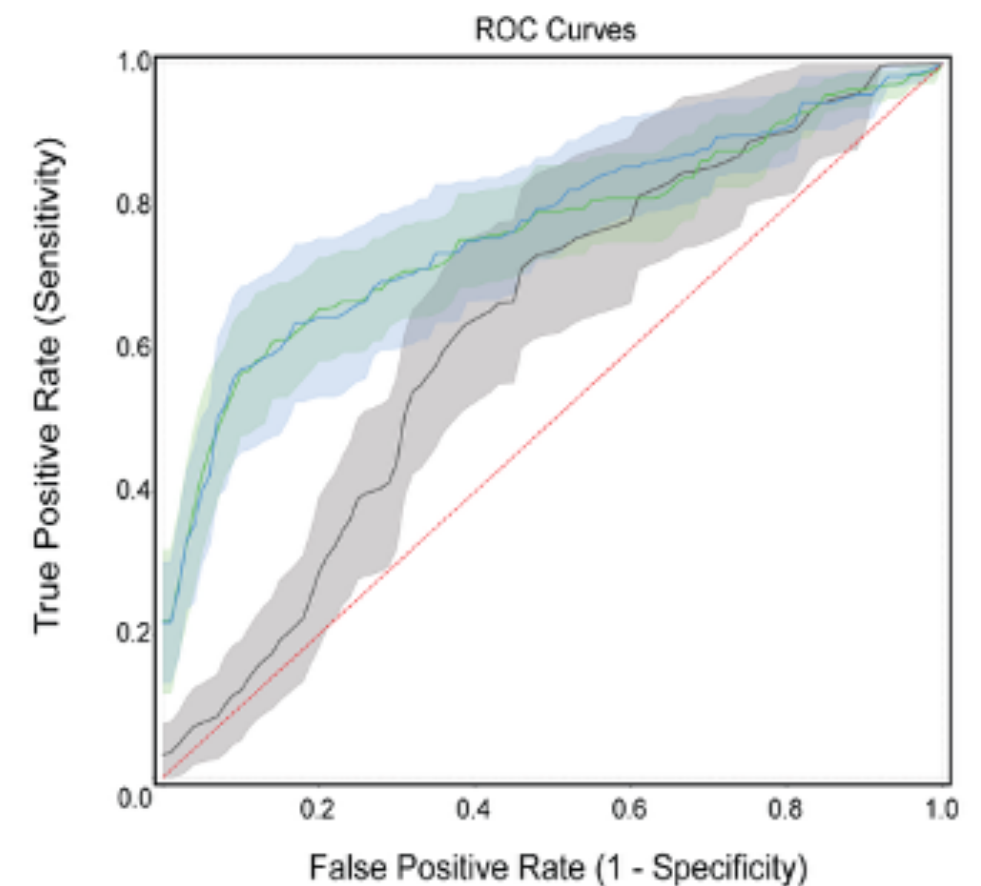
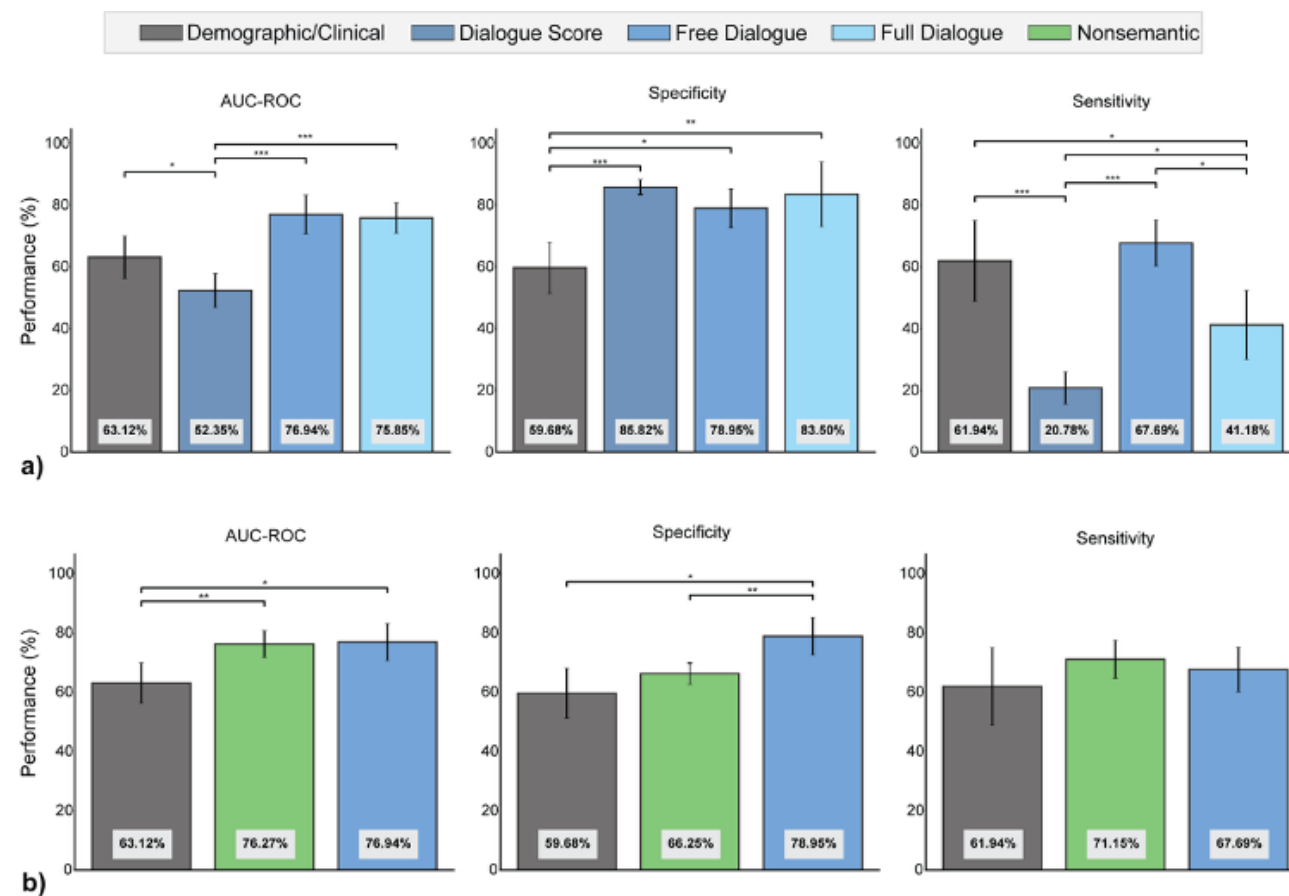
- Adicionalmente, de forma semántica se clasifica la 'representación' de cada entrevista



Visualización de la  
totalidad de la entrevista

# Emb: Resultados de comparación

- Cada modelo clasificó al paciente entre dos clases: positivo y negativo.
- Resultó mejor que los modelos basados en datos demográficos.



# Resumen

---

- Técnicas para representar texto: frecuencia, embeddings, transformers.
- Visualización de texto basada en frecuencia.
- Agrupamiento (clustering) y clasificación de temas basado en embeddings.
- Existen representaciones del lenguaje más complejas, como los transformers (donde las palabras tienen diferentes niveles semánticos y relaciones dentro de las oraciones). Si te interesa, comienza con BERT (o la versión en español BETO, realizada en el DCC)
- Todas las herramientas son útiles, usa la adecuada según cada caso.



*Book: Speech and Language Processing (Jurafsky)*